



Convenciones de Modelado

Convenciones de Modelado de la Cátedra de Diseño de Sistemas de Información

¿Qué son las convenciones de modelado?

Las convenciones de modelado en el software se refieren a un conjunto de reglas y pautas que quienes integran un equipo de trabajo siguen al diseñar modelos de software, ya sea a nivel de requerimientos, análisis, arquitectura, diseño y/o implementación.

Estas convenciones ayudan a establecer un vocabulario común entre los interesados, ya sea un equipo de desarrollo, una cátedra, una organización, que facilita la comprensión, mantenimiento y colaboración en el contexto del trabajo a realizar.

Se definen convenciones de modelado en el software, por ejemplo utilizando:

- ❖ Nomenclatura consistente: Utilizar nombres descriptivos y coherentes para clases, métodos, variables, etc., de modo que sea fácil entender su propósito y función.
- ❖ Diagramas de UML: Seguir las convenciones establecidas para el UML (Unified Modeling Language), como el uso adecuado de clases, interfaces, relaciones, etc.
- ❖ Patrones de diseño: Aplicar patrones de diseño reconocidos para resolver problemas comunes.
- ❖ Patrones GRASP: aplicar las tácticas recomendadas por los patrones GRASP a la hora de distribuir las responsabilidades entre las clases.
- ❖ Buenas prácticas de estilo al utilizar UML para modelar software, por ejemplo, la cantidad de elementos a incluir en una vista, el no cruzar líneas al modelar, etc.

¿Para qué sirven las convenciones de modelado?

La utilización de convenciones de modelado en el desarrollo de software proporciona una serie de beneficios y utilidades importantes para las personas involucradas y para la calidad del producto final. Algunas de las principales razones por las cuales es beneficioso usar convenciones de modelado son las siguientes:



Convenciones de Modelado

- ❖ **Facilita la comprensión:** Al seguir un conjunto de reglas y pautas comunes, se establece un estándar que facilita la comprensión de los modelos creados tanto para los participantes actuales del equipo como para participantes que requieran interpretar los modelos creados.
- ❖ **Mejora la comunicación:** Al adoptar convenciones de modelado, se establece un lenguaje común y una forma consistente de representar el software, lo que facilita la comunicación entre los miembros del equipo y con otras partes interesadas.
- ❖ **Aumenta la productividad:** Al seguir convenciones establecidas, se reduce la ambigüedad y se evitan malentendidos, lo que conduce a una mayor eficiencia y productividad en el proceso de desarrollo de software.
- ❖ **Facilita el mantenimiento:** La utilización de convenciones de modelado contribuye a la creación de software más mantenible, ya que modelos claros y coherentes facilitan la identificación y corrección de errores, así como la realización de modificaciones y actualizaciones en el futuro.
- ❖ **Promueve la consistencia:** Al mantener una estructura y estilo coherentes en los modelados del software, se promueve la consistencia en todo el sistema, lo que contribuye a su integridad y calidad global.
- ❖ **Ayuda en la escalabilidad:** Las convenciones de modelado bien establecidas pueden facilitar la escalabilidad del software, permitiendo que el sistema crezca de manera sostenible y se adapte a nuevas funcionalidades y requisitos sin comprometer su calidad.

En síntesis, el uso de convenciones de modelado en el desarrollo de software es fundamental para promover la claridad, la coherencia y la eficiencia en el proceso de creación de sistemas de software, lo que resulta en productos finales de mayor calidad y más fáciles de mantener y evolucionar.

¿Cuáles son las convenciones de modelado cubiertas en este documento?

Las convenciones de modelado que se presentan en este documento son las convenciones definidas por los docentes de la cátedra, sobre aspectos que no están definidos explícitamente en la bibliografía de referencia consultada.

En algunas ocasiones son simplificaciones adoptadas para modelar, en otras ocasiones el uso de expresiones lingüísticas, no obstante en ningún caso entran en contradicción con el paradigma de orientación a objetos, los principios de diseño, el UML, los lineamientos de estilo para el modelado con UML.



Convenciones de Modelado

Definición del Sistema de Información

Definición de alcances:

✍ Administrar: utilizamos es término para referirnos a la identificación a las funcionalidades relacionadas con los casos de uso mantenedores de información del sistema, conocidos como ABMC, es decir Registrar, Modificar, Consultar, Eliminar. Como ejemplo las **clases de soporte** son las que necesitan este tipo de casos de uso para administrarlas: TipoSuelo, TipoLaboreo, VariedadPizza, País, Provincia, Ciudad. También se utiliza el término administración para las **clases de definición** tales como: Cultivo, Agroquímico, Tratamiento. Ejemplo:

- Administrar cultivos, que abarca los casos de uso:

- ✍ Registrar Cultivo
- ✍ Modificar Cultivo
- ✍ Eliminar Cultivo
- ✍ Consultar Cultivo

✍ Gestionar: este término lo utilizamos para referirnos a los alcances asociados a las transacciones del dominio del problema. Ejemplo de transacciones dependiendo el dominio pueden ser: Venta, Cobro, Reserva, ProyectoDeCultivo, Pedido, etc. En este caso un alcance se escribiría:

- Gestionar reservas, que abarca los casos de uso:

- ✍ Registrar Reserva
- ✍ Modificar Reserva
- ✍ Consultar reserva
- ✍ Anular reservas no confirmadas
- ✍ Cancelar reservas



Convenciones de Modelado

Modelo de Casos de Uso

El modelo de casos de uso está conformado por varias vistas de casos de uso (construidas con diagramas de casos de uso), que pueden variar de acuerdo a lo que se desea mostrar, descripciones de casos de uso y descripciones de actores.

- ✍ **Definición de caso de uso:** cada caso de uso debe cumplir un objetivo y el nombre del caso de uso debe ser representativo de lo que hace el caso de uso.
 - **Se debe respetar el vocabulario del dominio del problema**, por ejemplo si en una pizzería el empleado refiere como **PEDIDO** a la transacción ocurrida cuando el cliente llega a la pizzería y solicita una o más pizzas, entonces los casos de uso asociados a la gestión del pedido se nombran usando el término pedido, por ejemplo: Registrar Pedido, Modificar Pedido, Cancelar Pedido, Consultar Pedido.
 - **El nombre del caso de uso debe darse desde la perspectiva del sistema:** es decir si estamos modelando un sistema que vende y cobra insumos de impresión, los casos de uso podrán llamarse: Registrar venta de insumo o Vender insumo; Cobrar insumos o registrar cobro de insumos que son os casos de uso relacionados con el cliente. Para los casos de uso relacionados con el proveedor serán: Registrar compra de insumos o Comprar insumos y Pagar insumos o registrar pago de insumos.
 - El nombre debe ser claro, aunque para alcanzar la claridad sea largo. Por ejemplo:
 - **Para un caso de uso de reporte será:** Generar reporte de cultivos con mayor rendimiento. Un caso de uso que se nombre como Generar reporte, no dice nada, se pueden generar muchos reportes en un sistema.
- ✍ **Casos de uso temporales:** pueden aparecer solos (sin actor) o identificar el tiempo como actor.
- ✍ **Casos de uso para administrar las clases de soporte o definición**, los casos de uso de tipo ABMC (altas, bajas, modificaciones, consultas) tal como se indicó en el apartado de alcance por ejemplo:
 - Administrar cultivos, que abarca los casos de uso:
 - Registrar Cultivo
 - Modificar Cultivo
 - Eliminar Cultivo
 - Consultar Cultivo

✍ Vista Esencial

La vista que se usa por lo general es la vista esencial, que es la vista que muestra los casos de uso que representan en forma directa a los objetivos del sistema de información y que consideran los alcances desde una perspectiva de escenario principal o feliz. Es decir sin considerar excepciones, errores o alternativas. Es decir:

- Si el alcance es **Administrar cultivos**, que abarca los casos de uso: Registrar Cultivo, Modificar Cultivo, Eliminar Cultivo, Consultar Cultivo; el caso de uso que se incluye en la vista esencial es el *Registrar Cultivo*, únicamente.
- No se incluyen casos de uso que representen cancelaciones, modificaciones.
- No se incluyen casos de uso genéricos como Iniciar sesión.
- Respecto a los reportes y/o estadísticas se incluyen algunos casos de uso representativos relacionados a los alcances.
- Pueden no estar las relaciones entre casos de uso (extensiones y/o inclusiones)
- No se incluyen los actores secundarios en especial si son de categoría software y/o hardware.



Convenciones de Modelado

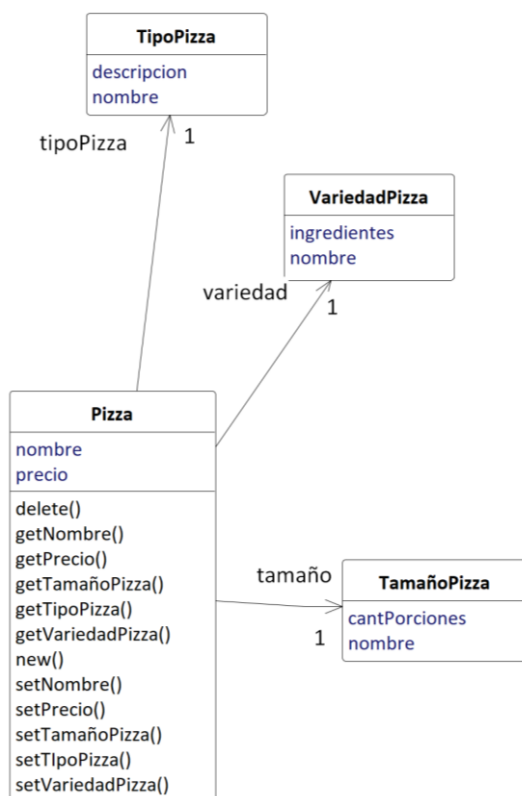
Modelado de Dominio

El modelo de dominio o modelo de dominio del problema lo construimos utilizando la herramienta diagrama de clases del UML.

Para esta vista que elegimos mostrar:

- ✍ Atributos, sin visibilidad ni tipo.
- ✍ Métodos, sin visibilidad, ni parámetros, ni retornos
- ✍ Atributos de referencia los modelamos sobre la relación en lugar de declararlo como atributo dentro de la clase.
- ✍ Respecto de los métodos de acceso o métodos de seteo, es decir get() y set() para cada atributo, al igual que el constructor al que denominamos "new()" y el destructor de objetos, que denominados "delete()" se especifican en una única clase, aclarando con una nota que es la clase donde se muestran estos métodos. Para el resto de las clases se asume que van, pero no se muestran.

Un ejemplo:

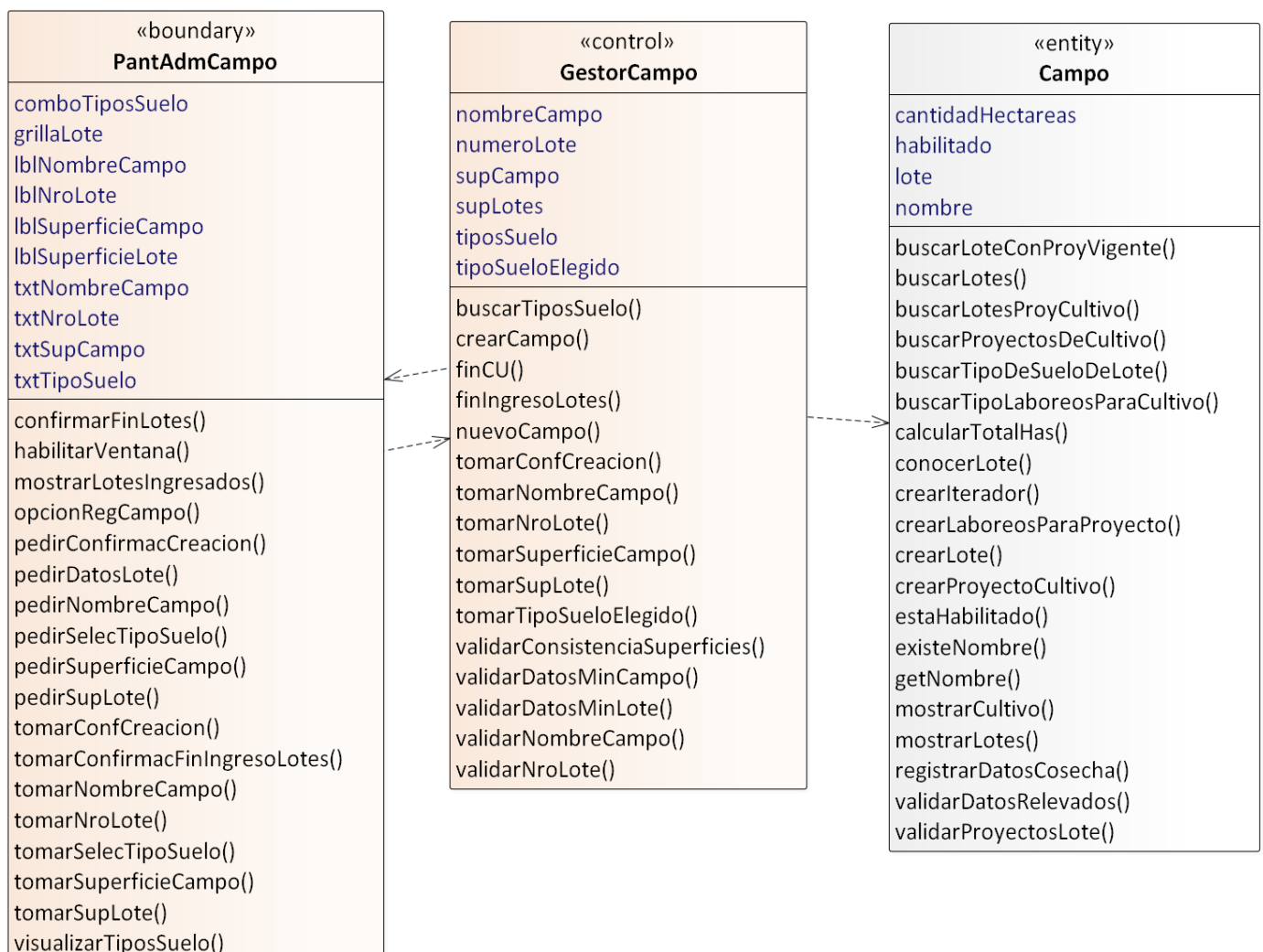




Convenciones de Modelado

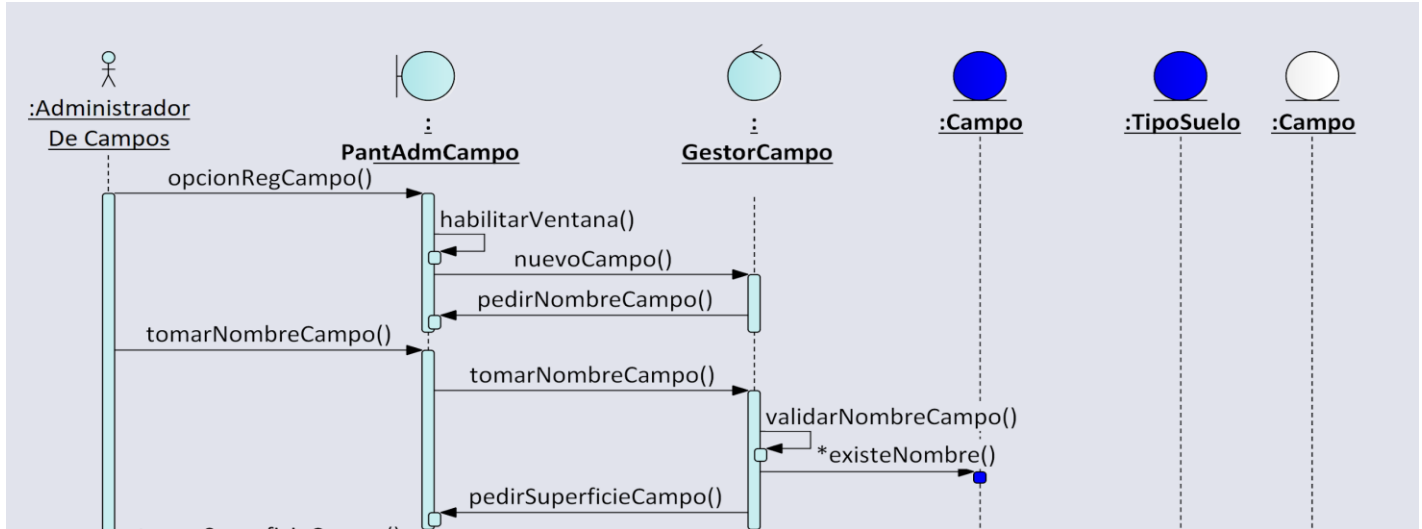
Realizaciones de Casos de uso de análisis

- ✍ En el contexto del análisis se modelarán los atributos, sin visibilidad ni tipo al igual que en el modelo de dominio.
- ✍ En el contexto del análisis se modelarán los métodos, sin visibilidad, ni parámetros, ni retornos.
- ✍ Se incluyen en la vista de estructura de la realización de CU de análisis lo siguiente:
 - Métodos en todas las clases únicamente los que se utilizan en la vista dinámica de la realización
 - Atributos en las clases de fabricación pura, si tiene, se incluyen al menos 5 que sean representativos de la realización que se está modelado.
 - Atributos de las clases de entidad: todos los que sean necesarios para el dominio planteado.
- ✍ **Relaciones de dependencia:** entre las clases de fabricación pura se modelará con relaciones de dependencia, al igual que entre las clases de control y las clases de entidad.
- ✍ En la vista estática de las realizaciones se modela con la representación etiquetada de la clase, según se muestra en la siguiente imagen:

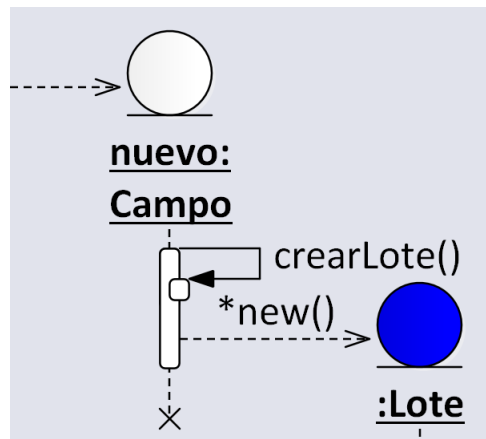


Convenciones de Modelado

✍ En la vista dinámica se muestra la representación icónica de los objetos, a saber:



✍ **Nombrado de objetos:** el nombrado de los objetos en UML se realiza poniendo un valor delante de los dos puntos, se recomienda que sea un valor significativo para lo que ese objeto representa. No es obligatorio nombrar a los objetos, en la cátedra lo hacemos cuando eso aporta información que mejora lo que el modelo quiere comunicar. Sí es importante recordar que solo se nombran instancias únicas, es decir cuando el objeto representa una única instancia. En el ejemplo que se muestra a continuación, está nombrado el objeto campo como "nuevo" porque se crea un campo, las instancias de lote no se nombran porque representa a todos los objetos lote que se crean, que pueden ser más de uno.



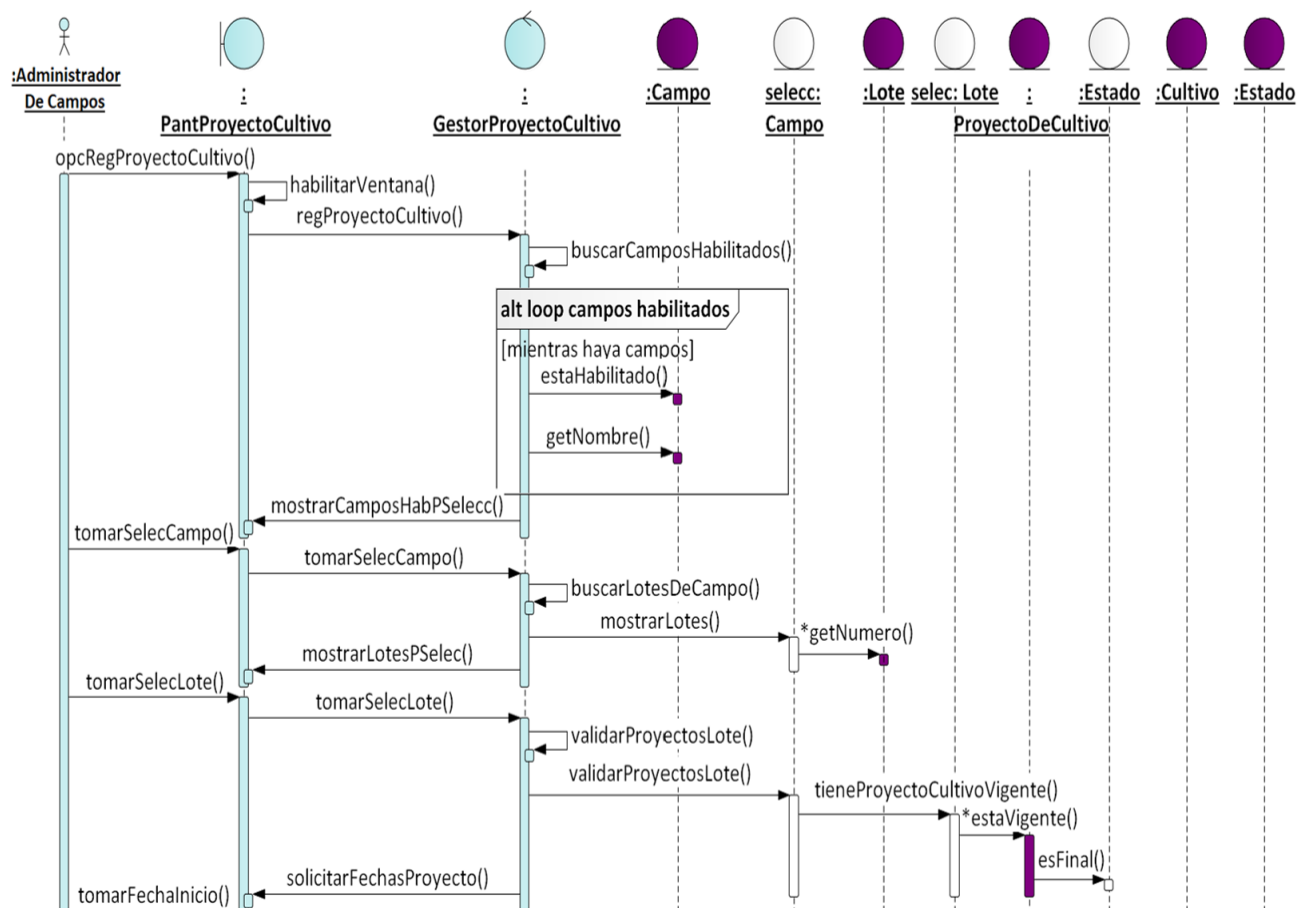
✍ **Métodos iteradores:** los asteriscos se utilizan en el modelado dinámico asociado a los métodos para indicar que ese método itera por todos los objetos a los que llega ese método. Se utiliza especialmente en el diagrama de comunicación, dado que en el diagrama de secuencia se utiliza para modelar fragmentos, con el operador de fragmento "loop". Como convención en la cátedra se modelará de la siguiente forma:

- Se utilizará el mismo objeto para representar todos los objetos de una clase, que para representar a un subconjunto que cumple con un criterio de selección.
- Los asteriscos en el diagrama de secuencia se utilizarán si es un único método el que forma parte de la iteración, de lo contrario se utilizará el fragmento de loop.

Convenciones de Modelado

- Se recomienda utilizar un color diferente para los objetos que representan a todos los objetos de una clase o a un subconjunto.

En la siguiente imagen se muestran lo explicado:





Convenciones de Modelado

Arquitectura Monolítica

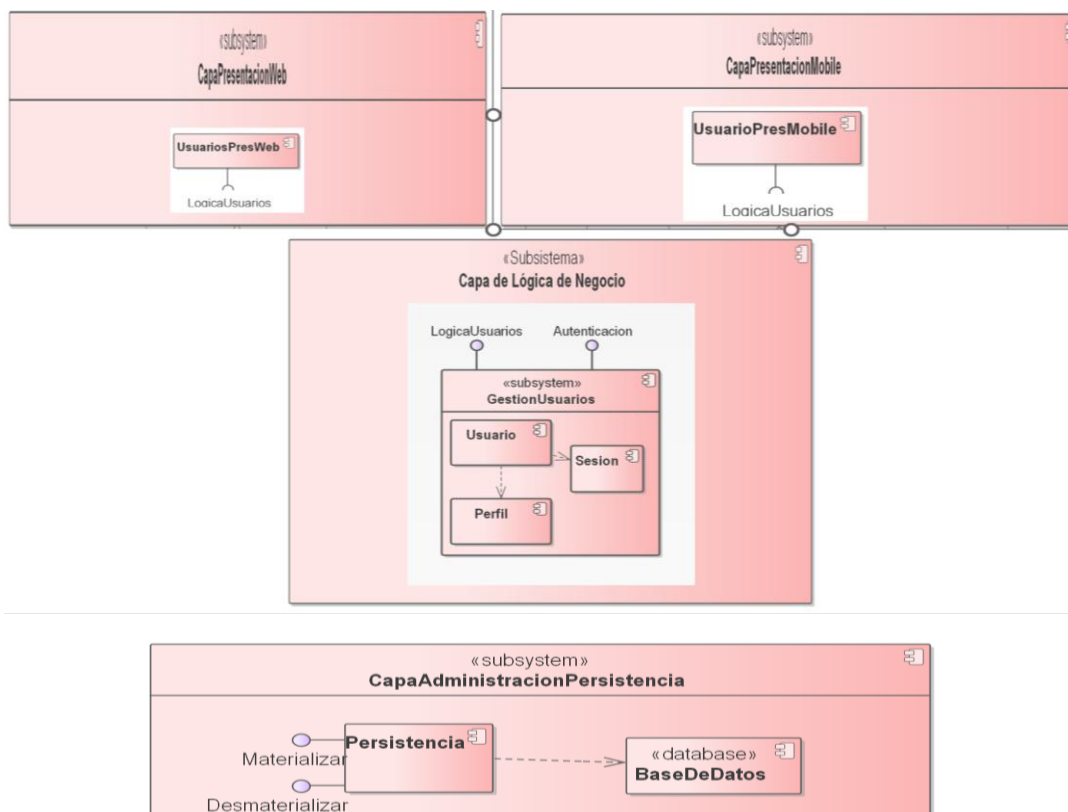
Convenciones para modelar la vista arquitectónica de la funcionalidad

- ✍ Se incluirán la menor cantidad de casos de uso que contemplen los requerimientos no funcionales significativos para la arquitectura y los requerimientos funcionales que la arquitectura debe resolver.
- ✍ Se complementará con una justificación de la inclusión de cada caso de uso.

Convenciones para modelar la vista arquitectónica del diseño

- ✍ En esta vista se incluirán únicamente los componentes que se desarrollarán, no los componentes de terceros.
- ✍ Se incluirán componentes representativos necesarios para resolver la funcionalidad expuesta en la vista arquitectónica de la funcionalidad.
- ✍ Interfaces se asocian al subsistema completo si es más de un componente el que provee o requiere lo expuesto en la interfaz, de lo contrario se asocia con el componente específico. El primer caso son las interfaces: LogicaUsuario y Autenticacion. El segundo caso son las interfaces Materialización y Desmaterialización.
- ✍ Con respecto a los componentes de presentación, se identificará un componente de presentación para cada tecnología por subsistema.

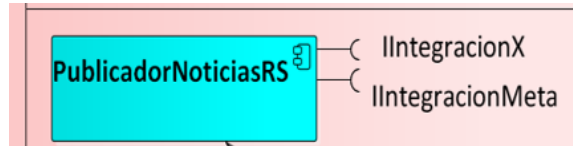
Por ejemplo:





Convenciones de Modelado

- ✍ Al modelar con interfaces expuestas debe coincidir el nombre de la interfaz provista con el de la interfaz requerida y en el único caso que puede haber una interfaz esté sola es cuando requiere servicios provistos por sistemas/servicios externos, por ejemplo:



Convenciones para modelar la vista arquitectónica del despliegue

- ✍ La cantidad de servidores para hacer el despliegue será la que esté explicitada, si no está explicitado, entonces se recomiendan las buenas prácticas de separar las capas de software en niveles de hardware diferentes.
- ✍ Para los nodos que representan servidores externos se espera que se identifiquen los componentes que se deducen explícitamente del enunciado, de lo contrario los servidores externos podrán quedar sin componentes.



Convenciones de Modelado

Arquitectura de Microservicios

Acuerdos y Convenciones para el modelado

Los acuerdos y convenciones propuestos en este documento son el resultado del trabajo realizado para la preparación del taller de Microservicios y surge como necesidad de acotar los contenidos que se abordarán en esta primera etapa de modelado en la cátedra y además la necesidad de tomar definiciones sobre aspectos de modelado que no están resueltos explícitamente en la bibliografía que se consultó.

✍ **Patrones para microservicios:** vamos a comenzar utilizando algunos, que los aplicaremos al diseño de las vistas.

✍ **Terminología:**

- Microservicio se llama el estilo arquitectónico. Cada una de las unidades funcionales dentro del producto es un “servicio” que lo representamos en UML con la notación de subsistema/componente con el estereotipo de microservicio.
- **Estilos vs. Patrones:** En la bibliografía que hemos tomado como referencia se menciona como Estilos a Monolítico y a Microservicios y patrones por ejemplo a estos que son los que vamos a utilizar hasta nuevo aviso:
 - Decompose by subdomain
 - Database per Service
 - API gateway
 - Backends for frontends
 - Single service per host
 - Multiple Services per host

✍ **Identificación de subdominios:** vamos a trabajar con el patrón de descomposición por subdominios, basado en DDD (Domain Driven Design) y vamos a modelar los subdominios clasificando por tipos de dominios, y esta identificación de dominios la usaremos de base para definir los servicios que tendrá el producto.

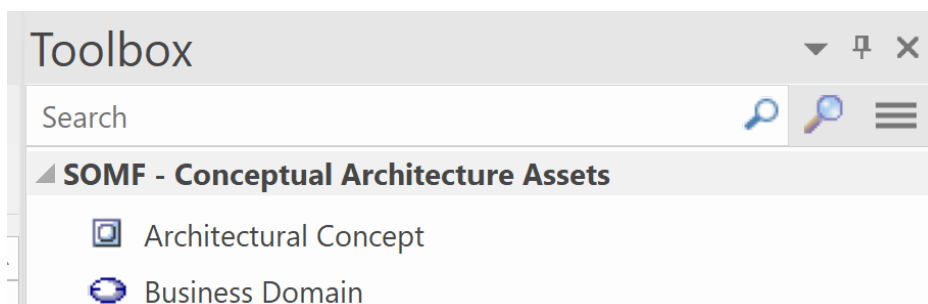
Los tipos de dominios son:

- Dominios Core: donde se realiza una inversión estratégica, prioritario. Distingue a la organización de la competencia.
- Dominios de Soporte: contribuyen al core facilitando que se pueda llevar a cabo. Son necesarios y tienen sentido en relación con el core y en ese sentido son específicos
- Dominios Genéricos: son aspectos que no son centrales del negocio, de hecho, son comunes a muchos tipos de negocio, y podrían incluso ser subcontratados, externalizados o comprados.

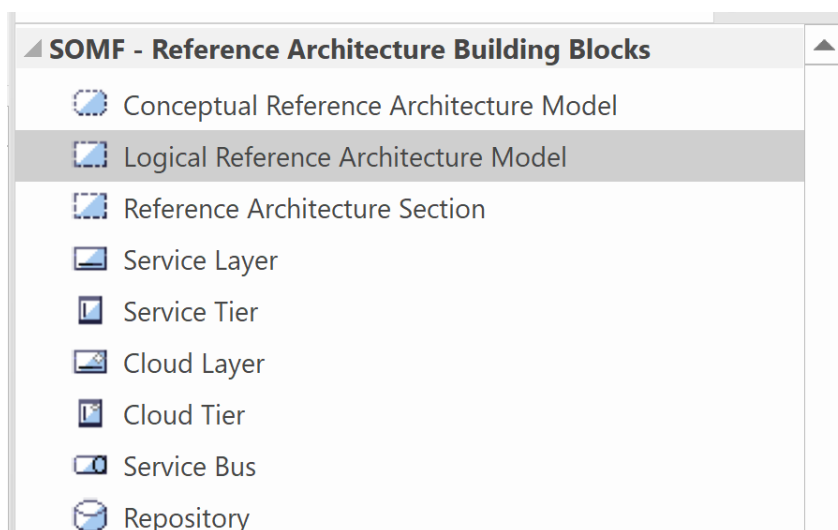
✍ **Modelado de los dominios en el EA:** para modelar los subdominios en el EA utilizamos dentro de la barra de herramientas la notación SOMF – Conceptual Architecture Assets, el elemento Business Domain:



Convenciones de Modelado



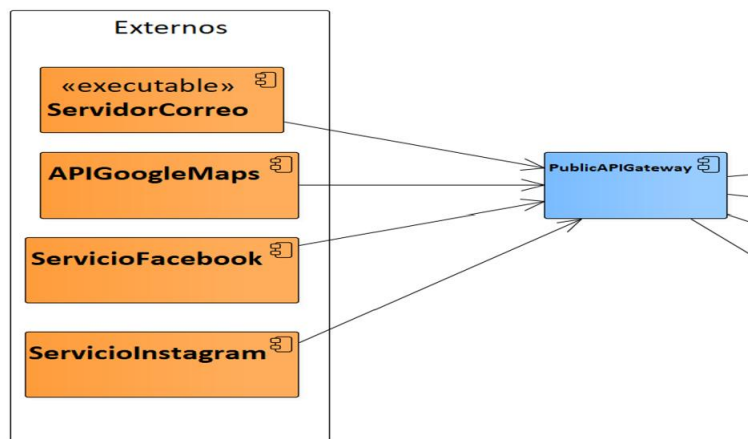
- ✍ **Modelado de las bases de datos en el EA:** para modelar las bases de datos en el EA utilizamos dentro de la barra de herramientas la notación SOMF – Reference Architecture Building Blocks, el elemento Repository.



- ✍ **Vistas a construir:**
- La vista de funcionalidad tal como se venía trabajando.
 - Se agrega la vista de diseño global para tener una imagen más general de la estructura de la aplicación.
 - La vista de diseño y de despliegue son las que se van a ver modificadas como resultado de este estilo arquitectónico y que se verán en detalle más adelante.
- ✍ **Convenciones para modelar en la Vista de Diseño Global**
- Los servicios provistos por terceros, externos, aparecen en esta vista y luego en la vista de despliegue, en la vista de diseño detallada no aparecen:



Convenciones de Modelado



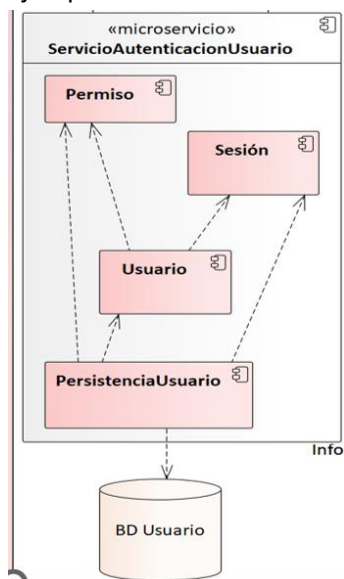
Convenciones para modelar la Vista de Diseño:

- Uso de API y/o API REST: en el glosario de la Cátedra en la UV hay definiciones respecto de estos recursos, la convención acordada en la cátedra es que cuando se necesita explicitar que se trabaja con comunicaciones asincrónicas optamos por API REST. Si no se explicita se puede usar API. Si bien la tendencia actual es hacia el uso de API REST.
- Vamos a trabajar microservicios para el backend con un único front, por ahora, microfrontends lo trabajaremos más adelante.
- Las interfaces de comunicación de los servicios que se comunican con el API Gateway o con el Backend for Frontend será por medio de API Rest. Esto privilegia las comunicaciones asincrónicas.
- Las interfaces de comunicación en los gateways son requeridas y la parte de front y los servicios tanto internos como externos tienen las interfaces provistas.
- Para la comunicación entre servicios internos del producto consideramos que los servicios de dominio genéricos son los que ofrecen, es decir tienen las interfaces provistas y analizamos quienes necesitan y ubicamos las interfaces requeridas.
- Para la comunicación con los servicios externos en la API Gateway Pública se ubican las interfaces requeridas, una por cada servicio externo, por ejemplo Servidor de Correo, Google Maps, MercadoPago, etc.; que en este caso quedarán sin interfaz provista. A su vez cada uno de los servicios que interactúan con los servicios externos (que por lo general son servicios de dominio genérico) se integrarán también a la API Gateway Pública.
- Las interfaces salen del servicio, que está encapsulado, no desde los componentes que hay dentro del servicio.
- La comunicación entre servicios de dominios core y dominios de soporte por intercambio de información o actualización de datos en las respectivas bases de datos no los vamos a modelar por ahora, se asume que la comunicación es por la API Rest de cada servicio que se conecta al API Gateway. Esto es una simplificación, de lo contrario deberíamos incorporar otros patrones (SAGA, CQRS, Messaging) y sería muy complejo.
- Respecto a la forma de estructurar cada servicio, recordar que la modularización de los servicios debe mantener la cohesión lo más alta posible. En ese sentido se recomienda que los componentes

Convenciones de Modelado

que manejan lógica de conversión o que dependen de sistemas externos no se ubiquen dentro de los servicios de tipo core.

- **Base de Datos y persistencia:** Vamos a trabajar con el patrón de una base de datos por servicio y cada servicio tiene un componente de persistencia que depende de los componentes que tiene que persistir y de la base de datos, por ejemplo:



✍ **Convenciones para modelar reportes y/o estadísticas:** se acordaron dos opciones:

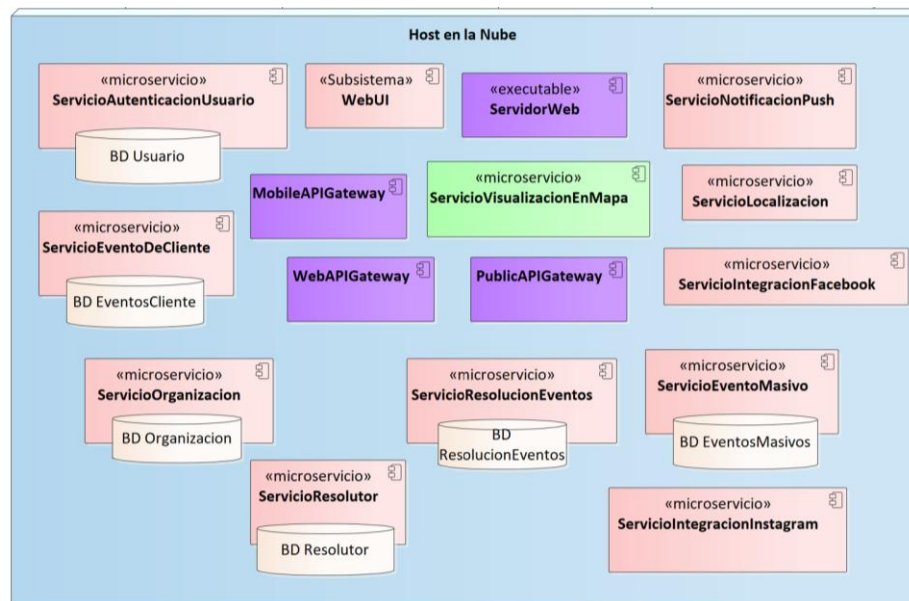
- **Opción 1:** Integración con una herramienta de reporteria externa como por ejemplo PowerBI, en este caso:
 - Debe estar especificado explícitamente como RNF.
 - Se definirá como un dominio genérico.
 - En la vista de diseño global incluirlo como componente externo y además un servicio de integración.
- **Opción 2:** Desarrollar los reportes y/o estadísticas internamente, en ese caso:
 - No aparece como dominio
 - No hay un servicio en la vista de diseño global.
 - En la vista de diseño aparecen interfaces desde los servicios core donde están los datos necesarios y la lógica de negocio desde donde generar los reportes y/o estadísticas.

✍ **Convenciones para modelar la Vista de Despliegue:**

- **Despliegue de las páginas web,** es decir los componentes del tipo "PresWeb" y el servidor web, se desplegarán:
 - Si usamos el patrón múltiples servicios por host, en el único servidor que tendremos. Por ejemplo:



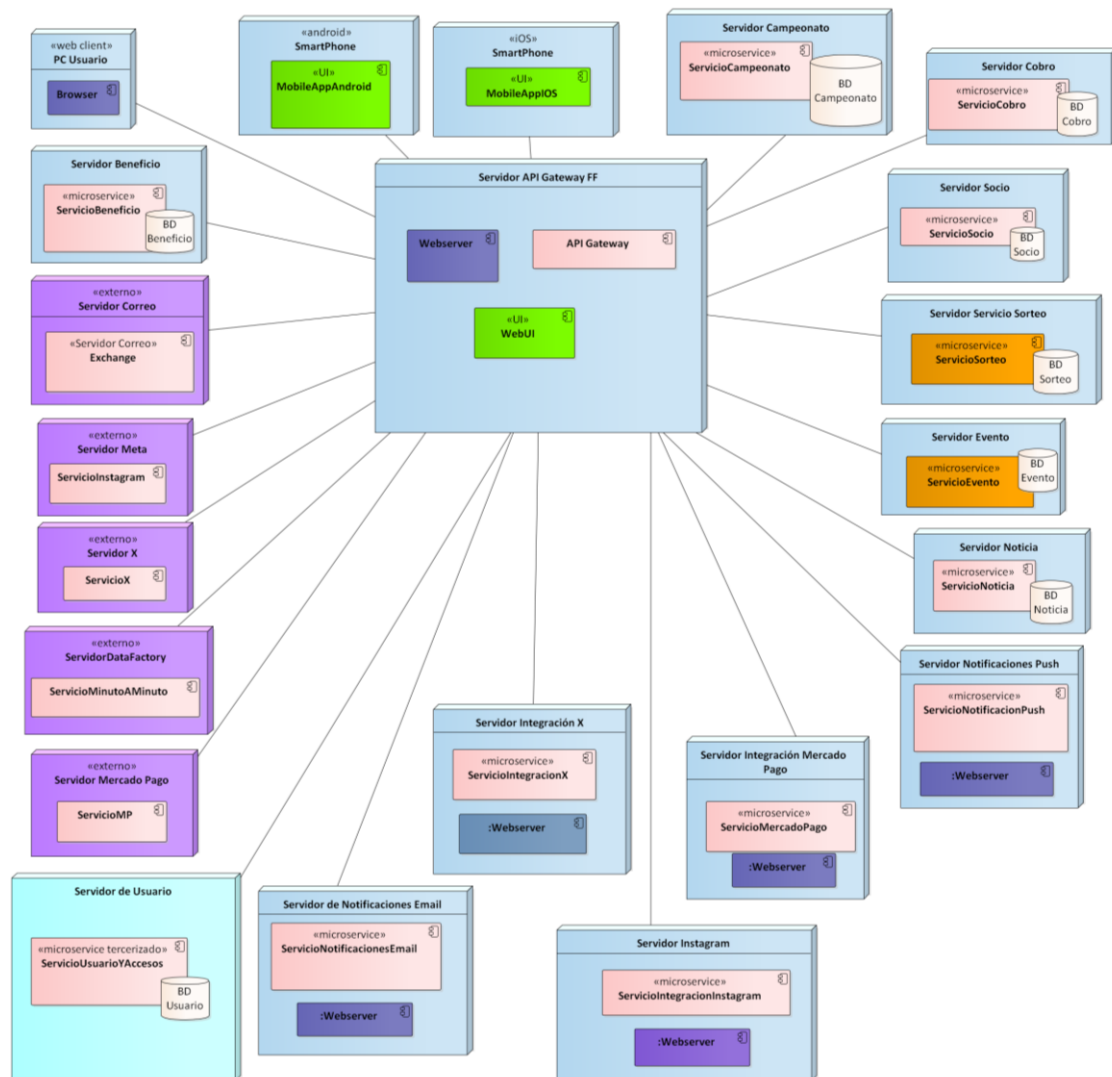
Convenciones de Modelado



- Si usamos el patrón un servicio por host, en el mismo servidor que alojamos el componente servidor web y el o los api gateways. También es posible replicar el servidor web en los servicios de dominios genéricos. Por ejemplo:



Convenciones de Modelado



- **Despliegue de componentes de UI Mobile**, en la vista de diseño se identifican los componentes (los mismos) y si se despliegan en SO Android y SO IOS, aparecen los dos clientes con los componentes repetidos.



Convenciones de Modelado

Historia de Cambios

Fecha	Versión	Descripción	Autor
30/03/2024	0.9	Versión inicial	Judith Meles
30/03/2024	1.0	Se agregó la convención respecto al nombrado de los objetos en las realizaciones de casos de uso	María Sanchez Judith Meles
06/04/2024	1.1	Se agregan ejemplos para despliegue de microservicios. Se agregan convenciones para modelado con casos de uso.	Judith Meles Cecilia Massano
27/07/2024	1.2	Se agregan convenciones para la vista de diseño de microservicios.	Judith Meles Cecilia Massano Elizabeth Jeinson